

Communication Game

Teacher Huang and his students are playing a game.

In this game, there are $N + 1$ students with IDs $0, 1, \dots, N$; the student with ID 0 is the class leader. The game consists of M rounds, numbered $0, 1, \dots, M - 1$. In round i , teacher Huang calls student $A[i]$ secretly into the room. These students remember each time they are called, but they are not informed how many rounds have passed, nor do they know which of the other students have entered previously.

During the game, the class leader is called into the room at most L times, and each other student is called into the room at most 2 times. The goal is for each student to correctly determine whether the class leader entered the room between their first and second visits.

To communicate, the students gather and decide on a common strategy. First, they will place K cards, numbered 0 to $K - 1$, in the room, each with black on one face and white on the other. Initially, all cards are placed with black facing up. In each round, the student enters the room and sees the colors on the cards, and they may flip them arbitrarily. For each student $1, 2, \dots, N$, after their second visit, they should immediately report whether the class leader entered the room between their first and second visits.

Your task is to devise a strategy for the students that ensures they answer all questions correctly. The score of your solution depends on the value of K (see Scoring section for details).

Implementation details

You need to implement the following four procedures:

```
int start_game(int L, int N, int i)
```

- L : the maximum number of times the class leader will be called into the room.
- N : the number of students, excluding the class leader.
- i : the ID of the student.
- This procedure should return the number K .
- The value of K should be the same for the same parameters L and N , regardless of i .
- The value of K must be between 0 and 64, inclusively.
- This procedure is called for at most 500 times per test case.

```
std::vector<bool> leader(std::vector<bool> c)
std::vector<bool> student_first(std::vector<bool> c)
std::pair<bool, std::vector<bool>> student_second(std::vector<bool> c)
```

- Procedure `leader` implements the class leader's strategy.
- Procedure `student_first` implements the other students' strategy for the first visit to the room.
- Procedures `student_second` implement the other students' strategy for the second visit to the room.
- c : array of length K . The i -th card's white face is up if and only if $c[i] = \text{true}$ when the student enters the room.
- The boolean arrays returned by the procedures should have length K . The i -th element is `true` if and only if the i -th card's white face is up after the student leaves the room.
- The first element of the pair returned by `student_second` should be whether the class leader entered the room between the student's first and second visits.
- All three procedures are called for at most 5000 times for each test case in total.

During the judging process, each process represents a single student. Each process will follow one of the three scenarios:

- The process only receives one call to `start_game` with $i = 0$ and subsequent calls to `leader`, for at most L times.
 - In this case, `leader` is called for round r , only if:
 - $A[r] = 0$,
 - `leader` has been called exactly once for every round $r' < r$ and $A[r'] = 0$, and
 - no other `leader` call for round $r' > r$ has been made.
- The process only receives one call to `start_game` with $i \neq 0$, at most one call to `student_first`, and at most one call to `student_second`, in this order.
 - If procedure `student_first` is called for round r , $A[r]$ is the first occurrence of i in the array.
 - If procedure `student_second` is called for round r , $A[r]$ is the second occurrence of i in the array, and procedure `student_first` must have been called before.
- The process never receives any call to the four procedures.

In other words, any process receiving a call to `start_game` will be called for every round r in a prefix of the rounds where $A[r] = i$ and no other calls will be made.

In addition to the constraints above, your program is required to be **deterministic**, that is, it should behave the same on the same call sequence and same parameters. Your program's behavior **can** and **should only** be determined by all calls to `start_game`, `leader`,

`student_first`, and `student_second`. In particular, each process will have its own global variables initialized normally.

Furthermore, in each test case, the program will be restarted for at most 500 times. Please note that overly large non-zero initialized global data will likely cause "Execution timed out" in this case.

Constraints

- $1 \leq L \leq 4095$.
- $1 \leq N \leq 128$.
- $1 \leq M \leq L + 2N$.
- The judge **is** adaptive; the value of M and the array A may change during the judging process as long as they are consistent to previous calls.

Scoring

Subtask	Score	Additional Constraints
1	6	$L = 1$.
2	14	$N \leq 64$.
3	80	No additional constraints.

In any of the test cases, if at least one of the following conditions hold, then the score of your solution for that test case will be 0 (reported as `Output isn't correct` in CMS):

- At least one of the length of the array returned is not K .
- The first element of the pair returned `student_second` is incorrect.
- The return value is inconsistent for the same call sequence with the same parameters. In other words, your program is nondeterministic.

Otherwise, in subtask 1 and 2, you will receive the full score. In subtask 3, your score is calculated as follows:

Condition	Score
$21 \leq K$	0
$13 \leq K \leq 20$	20
$9 \leq K \leq 12$	50
$K \leq 8$	80

Example

Consider the situation in which $L = 4, N = 2, M = 8, A = [1, 0, 2, 0, 2, 0, 1, 0]$.

The judge first spawns three processes, A, B, and C. Process A represents the class leader (student 0), process B represents student 1, and process C represent student 2.

Before the game starts, suppose the students' strategy is as follows.

- They need two cards ($K = 2$).
- The class leader always flips both cards for each round called into the room.
- On the first visit of other students, they flip their respective card (card 0 for student 1, card 1 for student 2), and memorize the color of the card facing up.
- On their second visit, if their assigned card is flipped to a different face, they deduce that the class leader must have been to the room during their two visits.

First, the following three calls are made:

Process	Procedure call	Return value
A	<code>start_game(2, 2, 0)</code>	2
B	<code>start_game(2, 2, 1)</code>	2
C	<code>start_game(2, 2, 2)</code>	2

According to the strategy, all there calls should return $K = 2$.

Now, the game starts. On round 0, $A[0] = 1$. The following call is made on process B for student 1:

```
student_first([0, 0]);
```

The student knows that he is student 1 from the previous call `start_game(2, 2, 1)`. Based on the strategy, the student flips the card 0. Hence, the procedure returns `[1, 0]`.

On round 1, $A[1] = 0$. The following call is made for student 0 on process A:

```
leader([1, 0]);
```

Based on the strategy, the class leader flips both cards, returning `[0, 1]`.

After that, the following calls are made:

Round	Student ID	Process	Procedure call	Return value
2	2	C	<code>student_first([0, 1])</code>	<code>[0, 0]</code>
3	0	A	<code>leader([0, 0])</code>	<code>[1, 1]</code>

On round 4, the student 2 is called into the room for the second time. The following call is made on process C:

```
student_second([1, 1]);
```

Student 2 learns that card 1 was black face up on his previous visit, but now it is white face up. Therefore, the class leader must have been to the room during his two visits. Hence, the procedure returns `(true, [1, 1])`.

Finally, the remaining rounds proceed as follows.

Round	Student ID	Process	Procedure call	Return value
5	0	A	<code>leader([1, 1])</code>	<code>[0, 0]</code>
6	1	B	<code>student_second([0, 0])</code>	<code>(true, [0, 0])</code>
7	0	A	<code>leader([0, 0])</code>	<code>[1, 1]</code>

The students answered all questions correctly. Thus, the test case is considered correct.

Sample Grader

Input format:

```
L N M
A[0]
A[1]
...
A[M-1]
```

The sample grader runs as follows:

- On the r -th run, the grader simulates up to the r -th round.
 - The grader first calls `start_game(L, N, A[r])`.
 - For each round $i = 0, 1, \dots, r - 1$:
 - If $A[i] \neq A[r]$, the sample grader infers the return value based on previous runs.

- If $A[i] = A[r]$, the grader calls the appropriate function, then checks if the return value is consistent with previous runs.
- For round r :
 - The grader calls the appropriate function and records the return value.
 - If $A[r] \neq 0$ and this is the second visit of the student to the room, the grader checks if the return value is correct.
- If less than M rounds have passed, restart the whole program.

It will always use `game.bin` in the current working directory to keep track of the sample grading process.

Notice that, with this strategy, the sample grader may have any of the following behaviors:

- Make more than 5000 calls to `leader`, `student_first`, and `student_second`.
- Make more than 500 calls to `start_game`.
- Restart the program more than 500 times.

The actual grader used in the judging process will not violate these constraints.

If the program is accepted by the sample grader, the sample grader prints

```
Accepted, K
```

where K is the declared K .

In addition, the grader prints the function call made for every run for debugging purposes.